

Berikut adalah penjelasan mendalam mengenai kode C# tersebut. Kode ini berfungsi untuk **membuat visualisasi grafik histogram** (diagram batang/garis) dari data gambar digital menggunakan pustaka GDI+ (System.Drawing).

Secara sederhana, kode ini menerima data frekuensi pixel (seberapa banyak warna gelap/terang) dan mengubahnya menjadi gambar grafik .png atau .bmp yang bisa ditampilkan di layar.

Penjelasan Per Blok Kode

Berikut adalah pembedahan kode tersebut baris demi baris dan blok demi blok:

1. Import Namespace

C#

```
using System.Drawing.Drawing2D;  
using System.Drawing;  
using System.Linq;
```

- **System.Drawing:** Menyediakan akses ke kelas dasar grafis seperti Bitmap, Graphics, Pen, Color.
- **System.Drawing.Drawing2D:** Menyediakan fitur grafis lanjutan, di sini digunakan untuk SmoothingMode (memperhalus garis).
- **System.Linq:** Digunakan untuk fungsi Max() pada array, guna mencari nilai tertinggi dalam histogram.

2. Definisi Kelas dan Metode

C#

```
namespace MiniPhotoshop.Logic.Histogram  
{  
    public static class HistogramDrawer  
    {  
        public static Bitmap Draw(int[] hist, int channel, int width, int height)  
        {
```

- **HistogramDrawer**: Kelas statis (tidak perlu di-*instantiate*) yang bertugas khusus menggambar.
- **Draw(...)**: Metode utama.
 - `int[] hist`: Array sepanjang 256 angka. Setiap angka mewakili jumlah pixel untuk tingkat kecerahan (0-255).
 - `int channel`: Menentukan warna garis (0=Merah, 1=Hijau, 2=Biru, dst).
 - `width, height`: Ukuran gambar output yang diinginkan.

3. Validasi dan Inisialisasi Canvas

C#

```
if (hist == null || hist.Length != 256) return new Bitmap(width, height);
```

```
Bitmap histBmp = new Bitmap(width, height);
```

```
using (Graphics g = Graphics.FromImage(histBmp))
{
    g.Clear(Color.Black);
    g.SmoothingMode = SmoothingMode.AntiAlias;
```

- **Validasi**: Jika data `hist` kosong atau panjangnya bukan 256 (standar histogram 8-bit), kembalikan gambar kosong agar program tidak *crash*.
- **new Bitmap(...)**: Membuat "kanvas" gambar kosong di memori.
- **Graphics.FromImage(...)**: Membuat objek `Graphics` (`g`) yang bertindak sebagai "pelukis" untuk menggambar di atas `histBmp`.
- **using (...)**: Memastikan memori untuk objek grafis dibersihkan segera setelah selesai dipakai.
- **g.Clear(Color.Black)**: Mewarnai seluruh latar belakang menjadi hitam.
- **SmoothingMode.AntiAlias**: Membuat garis diagonal atau lengkung terlihat halus (tidak bergerigi/kotak-kotak).

4. Persiapan Alat Gambar (Tools)

C#

```
Pen axisPen = new Pen(Color.White, 1);
```

```
Brush textBrush = Brushes.White;  
Font font = new Font("Arial", 8);
```

```
int marginLeft = 60;  
int marginBottom = 30;
```

- **axisPen:** Pena putih setebal 1 pixel untuk menggambar garis sumbu X dan Y.
- **textBrush:** Kuas putih untuk menulis angka (label).
- **font:** Jenis huruf Arial ukuran 8 untuk label.
- **Margin:** Variabel marginLeft dan marginBottom disiapkan untuk memberi jarak dari tepi gambar agar tulisan label tidak terpotong.

5. Menggambar Sumbu (Axis)

C#

```
// Gambar sumbu X dan Y  
g.DrawLine(axisPen, marginLeft, height - marginBottom, width - 10, height -  
marginBottom); // Sumbu X  
g.DrawLine(axisPen, marginLeft, 10, marginLeft, height - marginBottom); // Sumbu Y
```

- **Sumbu X (Horizontal):** Digambar dari kiri (marginLeft) ke kanan (width-10), pada posisi vertikal height - marginBottom. Ingat, dalam koordinat komputer, Y=0 ada di atas, jadi height - margin berarti posisi di bawah.
- **Sumbu Y (Vertikal):** Digambar dari atas (10) ke bawah (height - marginBottom) pada posisi horizontal marginLeft.

6. Logika Label Sumbu X (Nilai Pixel 0-255)

C#

```
int xAxisLength = width - marginLeft - 10;  
int labelCountX = 5;  
for (int i = 0; i <= labelCountX; i++)  
{  
    int val = i * 255 / labelCountX;  
    float xPos = marginLeft + (val / 255f) * xAxisLength;  
    float yPos = height - marginBottom;
```

```
g.DrawLine(Pens.White, xPos, yPos, xPos, yPos + 5);
```

```
string label = val.ToString();
```

```
SizeF size = g.MeasureString(label, font);
```

```
g.DrawString(label, font, textBrush, xPos - size.Width / 2, yPos + 5);
```

```
}
```

- **xAxisLength**: Menghitung panjang efektif garis sumbu X untuk menggambar data.
- **Loop**: Kode ini membagi sumbu X menjadi 5 bagian (titik 0, 51, 102, ... 255).
- **xPos**: Rumus konversi. Mengubah nilai 0-255 menjadi koordinat pixel layar.
 - Rumus: $\text{MarginKiri} + \left(\frac{\text{Nilai}}{255}\right) \times \text{PanjangGaris}$
- **DrawLine**: Membuat garis kecil (tick mark) ke bawah pada sumbu.
- **DrawString**: Menulis angka. $\text{size.Width} / 2$ digunakan agar teks berada tepat di tengah-tengah garis (center alignment).

7. Logika Label Sumbu Y (Frekuensi/Jumlah Pixel)

C#

```
int max = hist.Max();
```

```
if (max == 0) max = 1;
```

```
// ... (variabel labelCountY & margin)
```

```
for (int i = 0; i <= labelCountY; i++)
```

```
{
```

```
    int val = i * max / labelCountY;
```

```
    float yPos = height - marginBottom - (val / (float)max) * (height - 40);
```

```
g.DrawLine(Pens.White, marginLeft, yPos, marginLeft - 5, yPos);
```

```
string label = val.ToString();
```

```
SizeF size = g.MeasureString(label, font);
```

```
g.DrawString(label, font, textBrush, marginLeft - labelMarginRight - size.Width,  
yPos - size.Height / 2);
```

```
}
```

- **hist.Max()**: Mencari nilai tertinggi dalam data histogram. Ini penting untuk **Normalisasi**. Tanpa ini, grafik mungkin terlalu pendek atau menembus atap gambar.
- **if (max == 0)**: Mencegah error pembagian dengan nol (DivideByZeroException) jika gambar hitam total.

- **yPos**: Rumus konversi nilai frekuensi ke posisi pixel Y.
 - Karena koordinat Y komputer dimulai dari atas (0), kita harus mengurangi dari height.
- **DrawString**: Menulis angka di sebelah kiri sumbu Y, diratakan ke kanan (marginLeft - size.Width).

8. Memilih Warna Garis

C#

```
Pen pen;
switch (channel)
{
    case 0: pen = Pens.Red; break;
    case 1: pen = Pens.Lime; break; // Hijau terang
    case 2: pen = Pens.Blue; break;
    case 3: pen = Pens.White; break; // Biasanya untuk luminance/abu-abu
    default: pen = Pens.Gray; break;
}
```

- Berdasarkan parameter channel yang dikirim saat fungsi dipanggil, kode memilih warna pena untuk menggambar kurva histogram.

9. Menggambar Garis Histogram (Kurva Data)

C#

```
PointF[] points = new PointF[256];
for (int i = 0; i < 256; i++)
{
    float xPos = marginLeft + (i / 255f) * xAxisLength;
    float yPos = height - marginBottom - ((hist[i] / (float)max) * (height - 40));
    points[i] = new PointF(xPos, yPos);
}

g.DrawLine(pen, points);
}

return histBmp;
```

```

    }
}
}

```

- **PointF[] points:** Menyiapkan array titik koordinat sebanyak 256 titik.
- **Loop (0-255):**
 - Menghitung koordinat X untuk setiap level warna (0-255).
 - Menghitung koordinat Y berdasarkan jumlah pixel (`hist[i]`), diskalakan terhadap nilai maksimum (`max`) agar pas di dalam tinggi gambar.
- **g.DrawLine(pen, points):** Ini adalah perintah intinya. Ia menghubungkan ke-256 titik tersebut dengan garis lurus yang bersambung, membentuk kurva histogram.
- **return histBmp:** Mengembalikan hasil gambar ke pemanggil fungsi.

Ringkasan Fungsi

Kode ini mengubah **data mentah array** menjadi **gambar visual**. Jika array input berisi banyak angka di indeks 0-50, gambar yang dihasilkan akan memiliki grafik yang tinggi di sebelah kiri (menandakan gambar gelap). Jika angkanya merata, grafik akan terlihat datar.

Langkah selanjutnya:

Apakah Anda ingin saya menjelaskan bagaimana cara memanggil fungsi ini dari Form UI (User Interface) atau cara mendapatkan array `int[] hist` tersebut dari sebuah gambar?